Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИНЖЕНЕРНЫЙ БИЗНЕС И МЕНЕДЖМЕНТ»
КАФЕДРА «ПРОМЫШЛЕННАЯ ЛОГИСТИКА»

ОТЧЕТ
Предмет: Разработка интернет приложений
Лабораторные работы №5
“Взаимодействие клиентской и серверной частей”

Студент: Сигорский Александр Евгеньевич
Группа: ИБМ3-54Б
Преподаватель: М.В. Бурмистрова

Москва 2026

1. Описание задания

Цель лабораторной работы — организовать взаимодействие клиентской и серверной частей веб-приложения: отправлять HTTP-запросы из JavaScript, передавать параметры и JSON, получать ответы сервера, обрабатывать ошибки и использовать полученные данные для обновления интерфейса.

В CodeSlicer эта задача решается реальным модулем frontend/api-client.js. Браузерный интерфейс работает совместно с локальным Python API. В отличие от учебной схемы, где клиент может обращаться к единственному тестовому endpoint, здесь один API-клиент обслуживает состояние проекта, анализ, граф, review, инспекцию сущностей, адаптеры и инструменты.

CodeSlicer является локальной системой анализа программного кода. Он строит граф зависимостей и использует его для ответа на вопрос: какие части системы затронет изменение и почему. Визуально проект ориентируется на лаконичные интерфейсы современных AI-IDE, в том числе Kodik IDE, но логика взаимодействия полностью принадлежит CodeSlicer.

2. Схема взаимодействия

Общий путь данных выглядит следующим образом:

пользователь
→ HTML/JavaScript интерфейс
→ api-client.js
→ HTTP-запрос к local API
→ Python LocalApiHandler
→ анализ и состояние CodeSlicer
→ JSON-ответ
→ JavaScript
→ обновление экрана.

Клиент и сервер обслуживаются одним локальным приложением, поэтому запросы строятся как относительные пути: /api/health, /api/state, /api/graph, /api/review и другие. Это упрощает настройку и позволяет использовать same-origin модель.

3. Текст программы и пояснения

3.1. Универсальная функция request

Листинг 1:

```
async function request(path, options) {
    const response = await fetch(path, {
        cache: 'no-store',
        credentials: 'same-origin',
        ...options,
        headers: {
            ...(options?.headers || {})
        }
    });

    const text =
        await response.text();

    let data = {};

    try {
        data = text
            ? JSON.parse(text)
            : {};
    } catch (_) {
        data = {
            error: text
        };
    }

    if (path === '/api/health'
```

```
        && data.session_token) {
        sessionToken =
            data.session_token;
    }

    if (!response.ok) {
        const detail =
            data.error
            || data.message
            || data.status;

        const error =
            new Error(
                detail
                || response.statusText
            );

        error.status =
            response.status;

        error.payload = data;

        throw error;
    }

    return data;
}
```

Функция request является центральной точкой клиент-серверного обмена. Она получает URL и дополнительные параметры fetch. Настройка cache: 'no-store' говорит браузеру не использовать старый ответ для динамических данных. credentials: 'same-origin' разрешает передавать данные только в рамках того же origin.

Ответ сначала читается как текст. Это сделано намеренно: если backend вернёт некорректный JSON или обычный текст ошибки, интерфейс всё равно сможет сохранить содержимое и показать полезное сообщение.

3.2. Разбор JSON

Фрагмент:

```
const text = await response.text();
let data = {};

try {
   data = text
      ? JSON.parse(text)
      : {};
} catch (_) {
   data = {
      error: text
   };
}
```

Здесь отдельно разделены транспорт и разбор формата. Метод response.json() короче, но при невалидном JSON выбрасывает исключение сразу. Текущая реализация CodeSlicer сохраняет исходный текст, чтобы пользователь получил реальное сообщение backend.

3.3. Обработка HTTP-ошибок

Листинг 2:

```
if (!response.ok) {
   const detail =
      data.error
      || data.message
      || data.status;

   const error =
      new Error(
         detail
         || `${response.status}
```

```
            ${response.statusText}`
        );

    error.status =
        response.status;

    error.payload = data;

    throw error;
}
```

В JavaScript сам fetch не считает ответ 404 или 500 сетевой ошибкой. Поэтому приложение явно проверяет response.ok. Созданному Error добавляются status и payload. Благодаря этому вышестоящий код может различать разные классы ошибок и принимать решение в зависимости от ответа сервера.

Например, status 409 в CodeSlicer может означать, что операция требует локального подтверждения, а 422 — что переданные параметры не удалось обработать.

3.4. Отправка POST-запроса

Листинг 3:

```
const post = (path, payload) =>
    request(path, {
        method: 'POST',

        headers: {
            'Content-Type':
                'application/json',

            ...(sessionToken
                ? {
                    'X-CodeSlicer-Session':
                        sessionToken
```

```
            }
          : {})
    },

    body:
      JSON.stringify(
        payload || {}
      ),
  });
```

Функция post является обёрткой над request. Передаваемый JavaScript-объект преобразуется в JSON. Заголовок Content-Type позволяет Python backend понять формат тела запроса.

Если после /api/health был получен session_token, он добавляется как X-CodeSlicer-Session. Так браузер не должен самостоятельно придумывать идентификатор сессии: он получает его от локального сервера.

3.5. Готовые методы API-клиента

Листинг 4:

```
global.ImpactApi = {
  state:
    () =>
      request('/api/state'),

  graph:
    () =>
      request('/api/graph'),

  health:
    () =>
      request('/api/health'),

  adapters:
    () =>
```

```
        request('/api/adapters'),

    progress:
        () =>
        request('/api/progress'),

    inventory:
        () =>
        request('/api/inventory'),

    analyze:
        (project_path) =>
        post(
            '/api/analyze',
            { project_path }
        ),

    cancelAnalyze:
        () =>
        post(
            '/api/analyze/cancel',
            {}
        ),

    review:
        post.bind(
            null,
            '/api/review'
        ),
};
```

В остальном frontend не обязан каждый раз вручную писать fetch. Он использует семантические методы ImpactApi.health(), ImpactApi.state(), ImpactApi.analyze() и т. д. Это отделяет детали HTTP от прикладной логики интерфейса.

3.6. Пример запуска анализа с клиента

Пользователь вводит путь к проекту и нажимает кнопку анализа.
Обработчик получает значение input и передаёт его функции startAnalysis:

```
listen(
   '#analyzeButton',
   'click',
   () => startAnalysis(
      $('#projectPath').value
   )
);
```

Внутри процесса анализа используется метод:

```
await ImpactApi.analyze(
   projectPath
);
```

На backend этот вызов попадает в /api/analyze, где проверяется project_path и запускается LocalApiState.analyze. После выполнения клиент получает обновлённое состояние.

Таким образом, одно действие пользователя проходит через несколько уровней:
кнопка → event listener → функция интерфейса → API-клиент → HTTP POST → Python backend → анализ проекта → JSON → обновление DOM.

3.7. Получение карты проекта

Для карты используется POST-запрос с параметрами:

```
const response =
   await ImpactApi.graphProjection({
      project_path:
         state.project,

      level:
```

```
        state.mapLevel,

      query:
        $('#graphProjectionQuery')
          ?.value.trim()
        || '',

      filters,

      show_all:
        state.mapScope
          === 'all',

      max_nodes: 120,
      max_edges: 200
    });

state.projection = response;
renderProjection(response);
```

В тело запроса помещаются настройки представления. Backend не обязан возвращать весь огромный граф: он строит ограниченную проекцию, подходящую для текущего режима. Полученный объект сохраняется в состоянии интерфейса и визуализируется.

3.8. Инициализация связи при запуске страницы

Листинг 5:

```
async function bootstrap() {
  try {
    const [
      health,
      current
    ] =
      await Promise.all([
        ImpactApi.health(),
```

```
            ImpactApi.state()
        ]);

      if (health.status !== 'ok') {
        throw new Error(
          'local API не готов'
        );
      }

      state.project =
        current.project_path
        || '';

      state.hasAnalysis =
        Boolean(
          current.has_analysis
        );

      state.analysis =
        current.analysis
        || null;

      state.ready = true;

      renderRoute();
    } catch (error) {
      state.ready = true;

      renderRoute();

      showState(
        `Локальный API недоступен:
        ${error.message}`,
        'error-state'
      );
    }
}
```

При старте интерфейс не считает backend доступным заранее. Он делает реальный health-check и только после успешного ответа переводит приложение в рабочее состояние. Если соединение не установлено, пользователь видит ошибку.

3.9. Локальное подтверждение чувствительных действий

Для отдельных действий backend отвечает состоянием pending_approval. API-клиент распознаёт такой ответ и запрашивает одноразовый локальный токен.

Листинг 6:

```
async function executeApprovedAction(
    callback,
    payload
) {
    const exactPayload =
        Object.freeze({
            ...(payload || {})
        });

    try {
        return await callback(
            exactPayload
        );
    } catch (error) {
        const pending =
            error
            && error.status === 409
            && error.payload
            && error.payload.status
                === 'pending_approval'
                ? error.payload
                : null;
```

```
      if (!pending) {
        throw error;
      }

      const token =
        await requestApprovalToken(
          pending
        );

      if (!token) {
        throw error;
      }

      return callback({
        ...exactPayload,
        approval_id:
          pending.approval
            ?.approval_id,
        approval_token:
          token
      });
    }
}
```

Этот код показывает более сложный клиент-серверный сценарий. Первый запрос может не выполнить действие, а вернуть данные о необходимости подтверждения. После ввода токена выполняется повторный запрос с тем же исходным payload и дополнительными полями.

4. Экранные формы с примерами выполнения программы

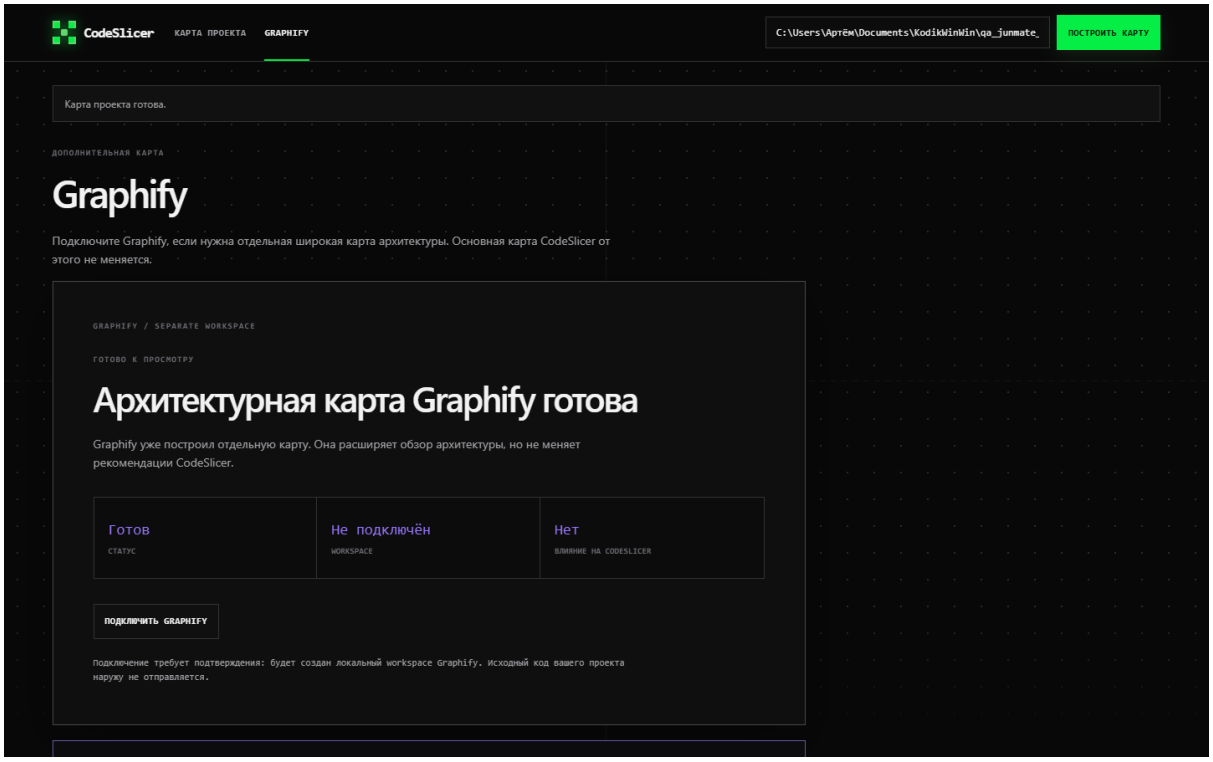


Рисунок 1 — карта CodeSlicer, полученная клиентом от local API.

Интерфейс строит визуализацию не из заранее записанного демонстрационного набора, а из ответа backend для выбранного проекта.

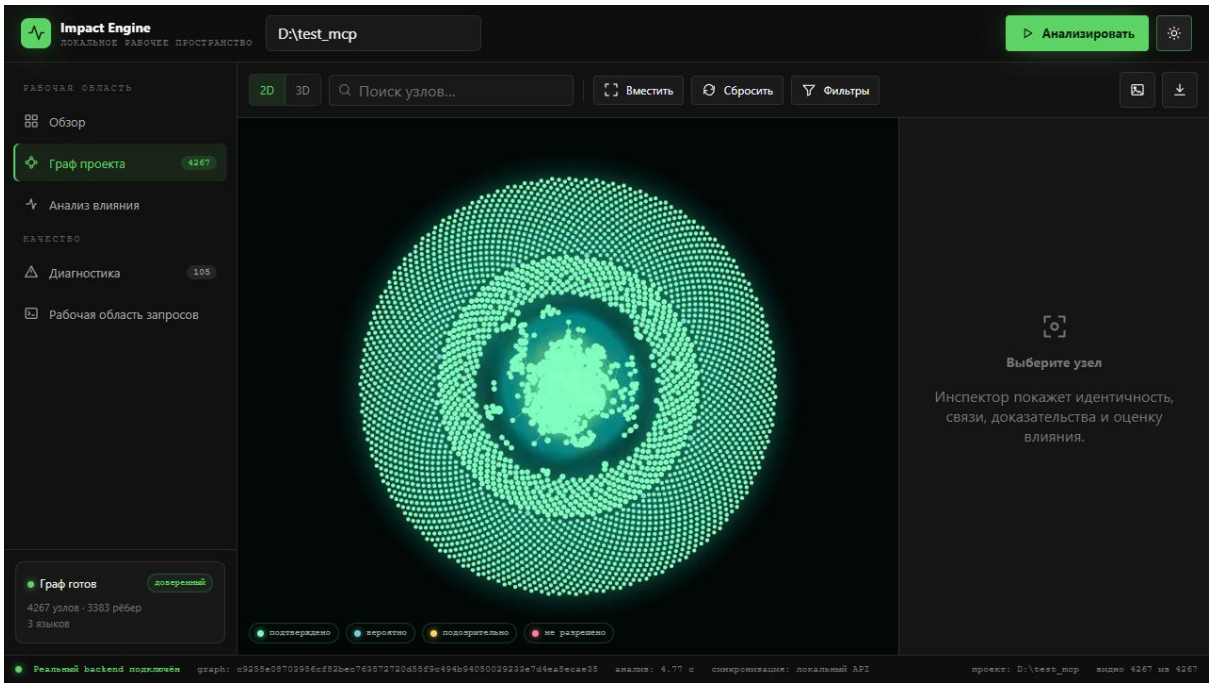


Рисунок 2 — Граф проекта CodeSlicer

Экран является результатом цепочки клиент-серверного обмена: JavaScript формирует запрос, backend возвращает структуру графа, после чего frontend строит элементы визуализации.

Проверка изменений также выполняется через API. В ответ клиент получает структурированный отчёт: затронутые файлы и сущности, риск, рекомендации и сведения о доказательствах.

5. Вывод

В лабораторной работе рассмотрено реальное взаимодействие клиентской и серверной частей CodeSlicer. Центральный модуль api-client.js инкапсулирует работу с fetch, преобразование JSON, проверку HTTP-статусов и передачу параметров.

Практически важным является разделение ответственности. Код интерфейса вызывает ImpactApi и работает с объектами предметной области, а детали HTTP находятся в одном модуле. Backend, в свою очередь, проверяет входные параметры, выполняет операции и возвращает однозначные JSON-ответы.

В текущей версии CodeSlicer используется Fetch API, а не устаревающий XMLHttpRequest. При этом выполняется та же базовая задача, которая требуется от клиент-серверной лабораторной работы: асинхронно отправить запрос, получить ответ, обработать ошибку и отобразить результат пользователю.